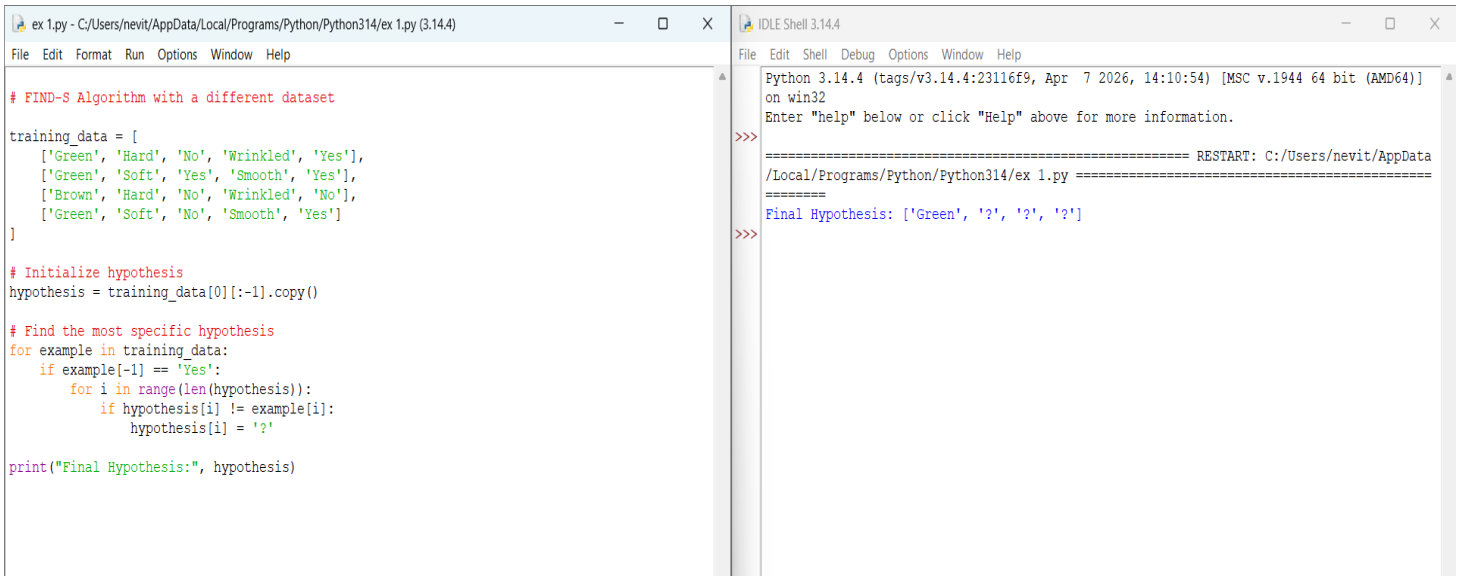
Lab Experiments

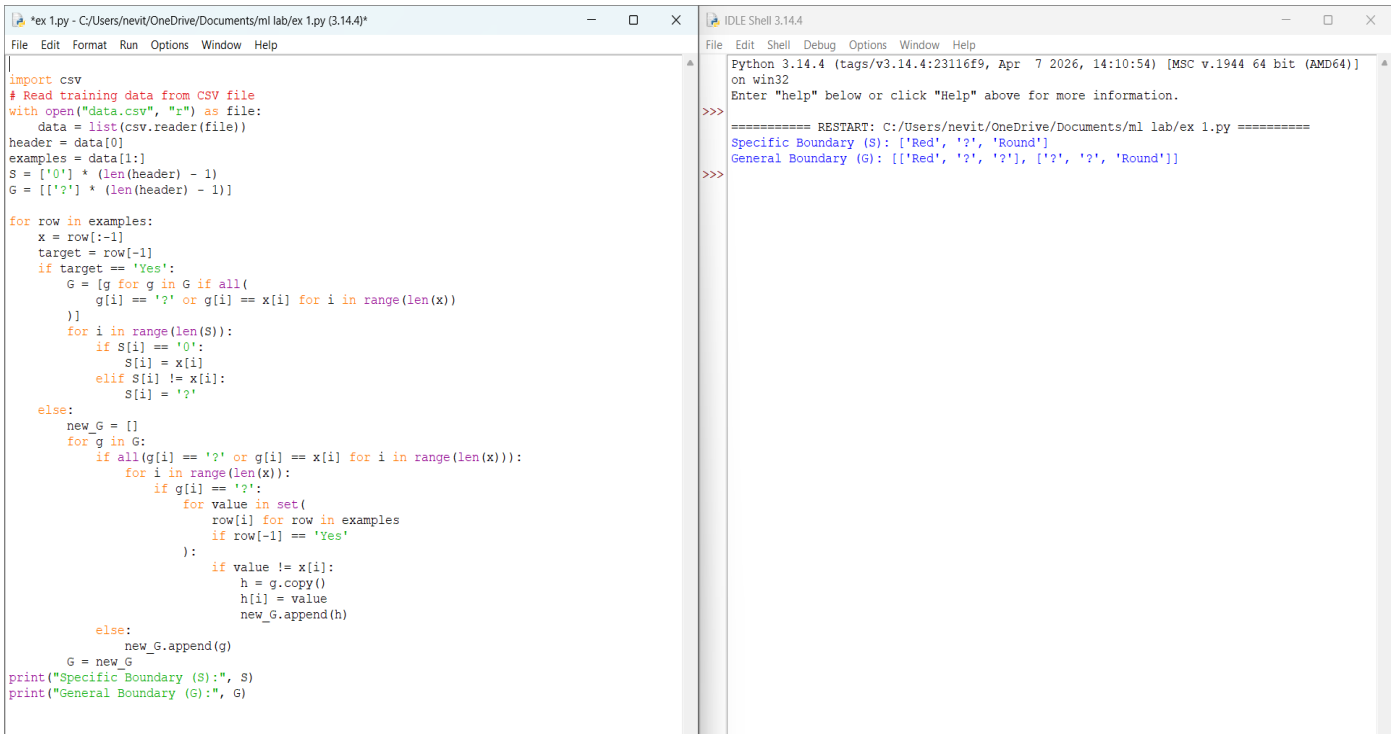
Course Code:ITA-0610 Name:Nevitha M

Course Name: Machine Learning Reg no:192521206

1. Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.



2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

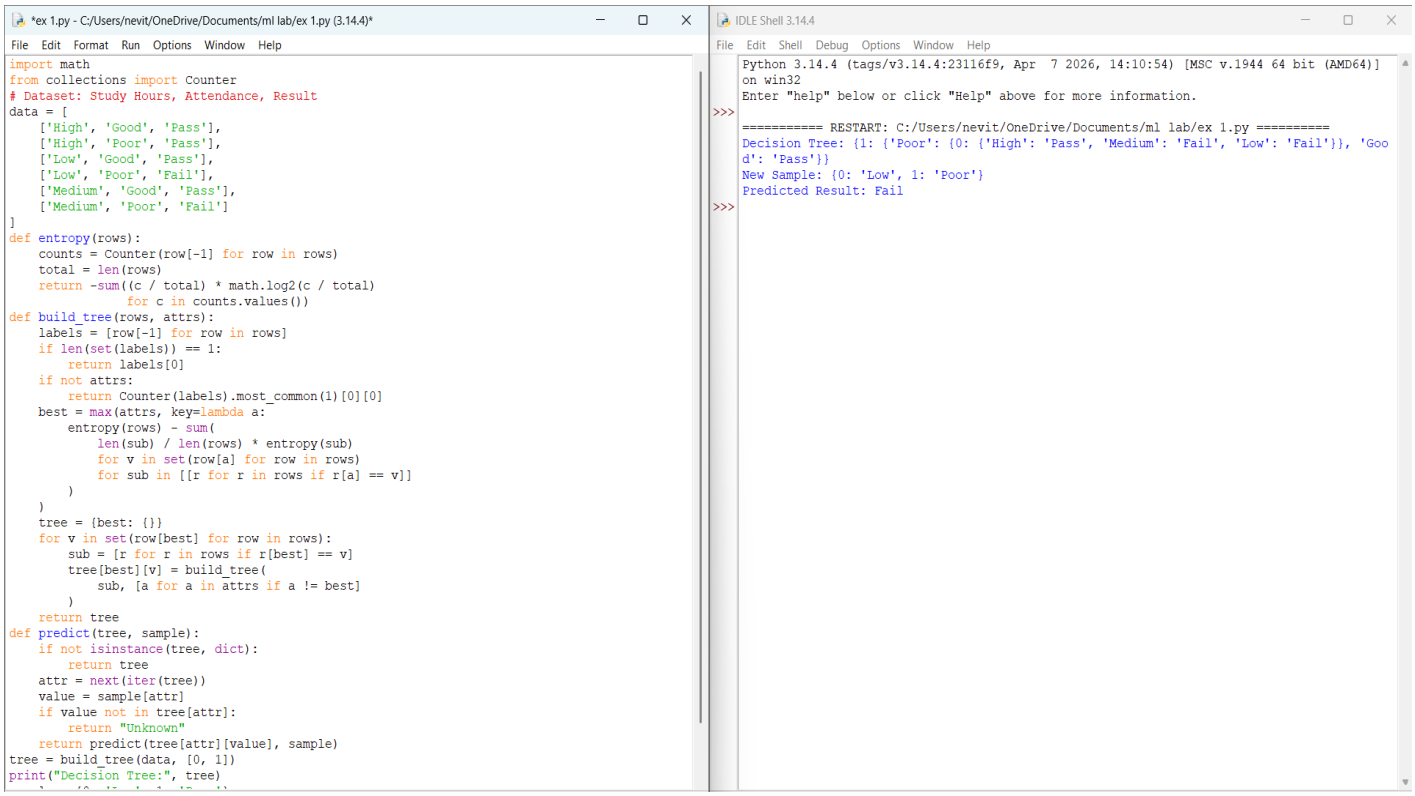


```
*ex 1.py - C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py (3.14.4)*
File  Edit  Format  Run  Options  Window  Help
import csv
# Read training data from CSV file
with open("data.csv", "r") as file:
    data = list(csv.reader(file))
header = data[0]
examples = data[1:]
S = ['0'] * (len(header) - 1)
G = [['?'] * (len(header) - 1)]

for row in examples:
    x = row[:-1]
    target = row[-1]
    if target == 'Yes':
        G = [g for g in G if all(
            g[i] == '?' or g[i] == x[i] for i in range(len(x))
        )]
        for i in range(len(S)):
            if S[i] == '0':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?'
    else:
        new_G = []
        for g in G:
            if all(g[i] == '?' or g[i] == x[i] for i in range(len(x))):
                for i in range(len(x)):
                    if g[i] == '?':
                        for value in set(
                            row[i] for row in examples
                            if row[-1] == 'Yes'
                        ):
                            if value != x[i]:
                                h = g.copy()
                                h[i] = value
                                new_G.append(h)
            else:
                new_G.append(g)
        G = new_G
print("Specific Boundary (S):", S)
print("General Boundary (G):", G)
```

```
IDLE Shell 3.14.4
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py ==========
Specific Boundary (S): ['Red', '?', 'Round']
General Boundary (G): [['Red', '?', '?'], ['?', '?', 'Round']]
>>>
```

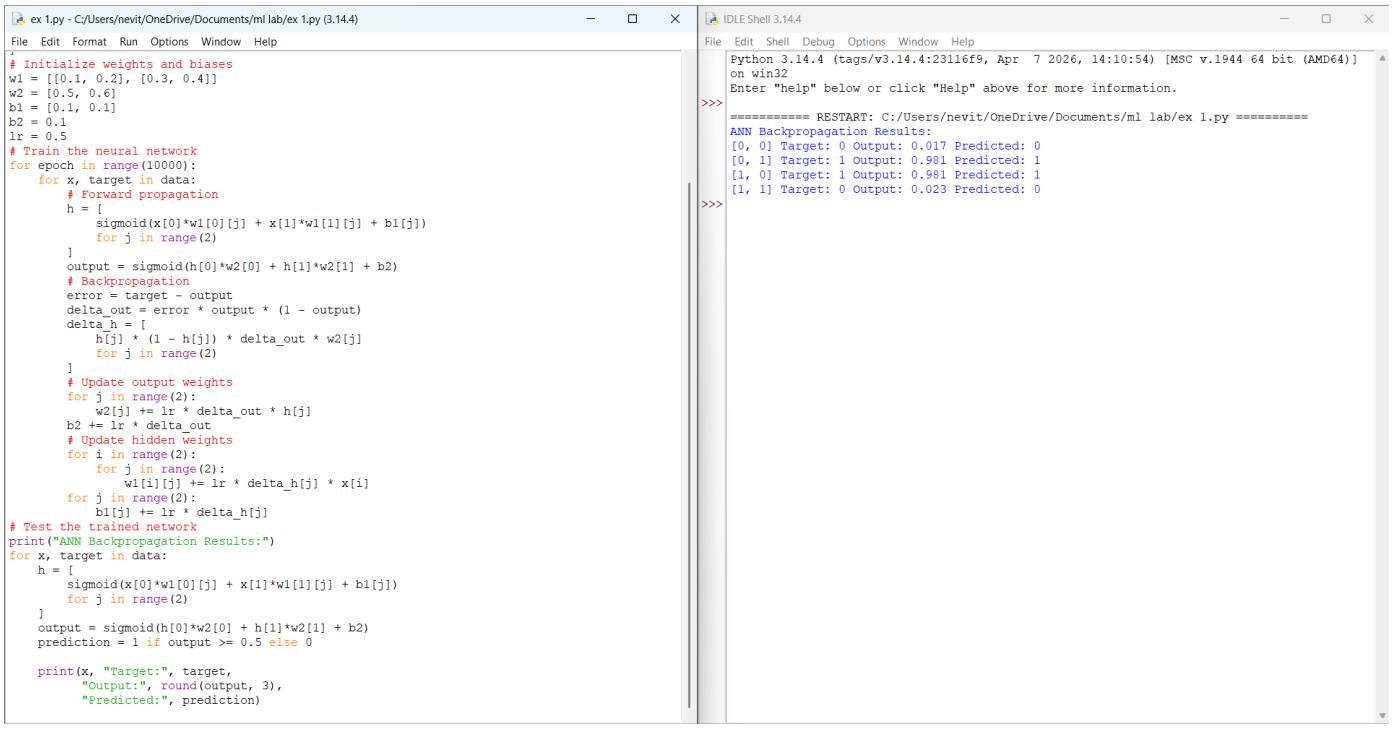
3. Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.



```
*ex 1.py - C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py (3.14.4)*
File  Edit  Format  Run  Options  Window  Help
import math
from collections import Counter
# Dataset: Study Hours, Attendance, Result
data = [
    ['High', 'Good', 'Pass'],
    ['High', 'Poor', 'Pass'],
    ['Low', 'Good', 'Pass'],
    ['Low', 'Poor', 'Fail'],
    ['Medium', 'Good', 'Pass'],
    ['Medium', 'Poor', 'Fail']
]
def entropy(rows):
    counts = Counter(row[-1] for row in rows)
    total = len(rows)
    return -sum((c / total) * math.log2(c / total)
                for c in counts.values())
def build_tree(rows, attrs):
    labels = [row[-1] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not attrs:
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda a:
        entropy(rows) - sum(
            len(sub) / len(rows) * entropy(sub)
            for v in set(row[a] for row in rows)
            for sub in [[r for r in rows if r[a] == v]]
        )
    )
    tree = {best: {}}
    for v in set(row[best] for row in rows):
        sub = [r for r in rows if r[best] == v]
        tree[best][v] = build_tree(
            sub, [a for a in attrs if a != best]
        )
    return tree
def predict(tree, sample):
    if not isinstance(tree, dict):
        return tree
    attr = next(iter(tree))
    value = sample[attr]
    if value not in tree[attr]:
        return "Unknown"
    return predict(tree[attr][value], sample)
tree = build_tree(data, [0, 1])
print("Decision Tree:", tree)
```

```
IDLE Shell 3.14.4
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py ==========
Decision Tree: {1: {'Poor': {0: {'High': 'Pass', 'Medium': 'Fail', 'Low': 'Fail'}}, 'Goo
d': 'Pass'}}
New Sample: {0: 'Low', 1: 'Poor'}
Predicted Result: Fail
>>>
```

4. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.



ex 1.py - C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py (3.14.4)

File Edit Format Run Options Window Help

```
# Initialize weights and biases
w1 = [[0.1, 0.2], [0.3, 0.4]]
w2 = [0.5, 0.6]
b1 = [0.1, 0.1]
b2 = 0.1
lr = 0.5
# Train the neural network
for epoch in range(10000):
    for x, target in data:
        # Forward propagation
        h = [
            sigmoid(x[0]*w1[0][j] + x[1]*w1[1][j] + b1[j])
            for j in range(2)
        ]
        output = sigmoid(h[0]*w2[0] + h[1]*w2[1] + b2)
        # Backpropagation
        error = target - output
        delta_out = error * output * (1 - output)
        delta_h = [
            h[j] * (1 - h[j]) * delta_out * w2[j]
            for j in range(2)
        ]
        # Update output weights
        for j in range(2):
            w2[j] += lr * delta_out * h[j]
        b2 += lr * delta_out
        # Update hidden weights
        for i in range(2):
            for j in range(2):
                w1[i][j] += lr * delta_h[j] * x[i]
        for j in range(2):
            b1[j] += lr * delta_h[j]
# Test the trained network
print("ANN Backpropagation Results:")
for x, target in data:
    h = [
        sigmoid(x[0]*w1[0][j] + x[1]*w1[1][j] + b1[j])
        for j in range(2)
    ]
    output = sigmoid(h[0]*w2[0] + h[1]*w2[1] + b2)
    prediction = 1 if output >= 0.5 else 0

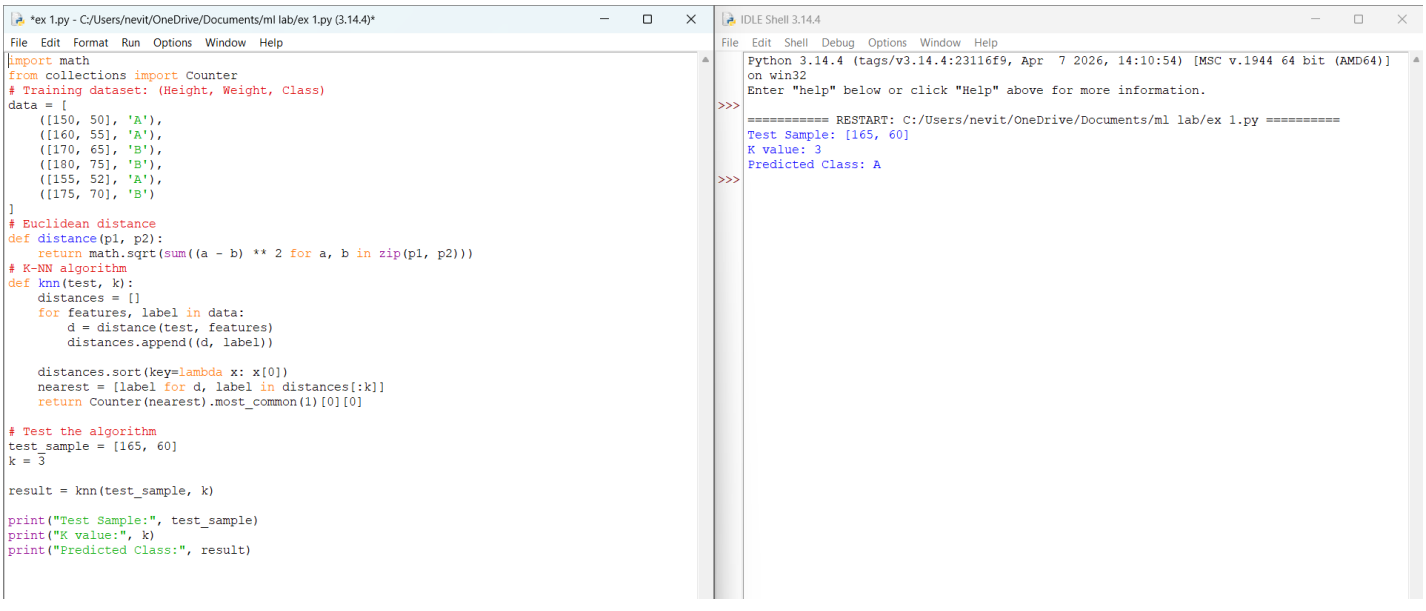
    print(x, "Target:", target,
          "Output:", round(output, 3),
          "Predicted:", prediction)
```

IDLE Shell 3.14.4

File Edit Shell Debug Options Window Help

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py ==========
ANN Backpropagation Results:
[0, 0] Target: 0 Output: 0.017 Predicted: 0
[0, 1] Target: 1 Output: 0.981 Predicted: 1
[1, 0] Target: 1 Output: 0.981 Predicted: 1
[1, 1] Target: 0 Output: 0.023 Predicted: 0
>>>
```

5. Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python



ex 1.py - C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py (3.14.4)

File Edit Format Run Options Window Help

```
import math
from collections import Counter
# Training dataset: (Height, Weight, Class)
data = [
    ([150, 50], 'A'),
    ([160, 55], 'A'),
    ([170, 65], 'B'),
    ([180, 75], 'B'),
    ([155, 52], 'A'),
    ([175, 70], 'B')
]
# Euclidean distance
def distance(p1, p2):
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))
# K-NN algorithm
def knn(test, k):
    distances = []
    for features, label in data:
        d = distance(test, features)
        distances.append((d, label))

    distances.sort(key=lambda x: x[0])
    nearest = [label for d, label in distances[:k]]
    return Counter(nearest).most_common(1)[0][0]

# Test the algorithm
test_sample = [165, 60]
k = 3

result = knn(test_sample, k)

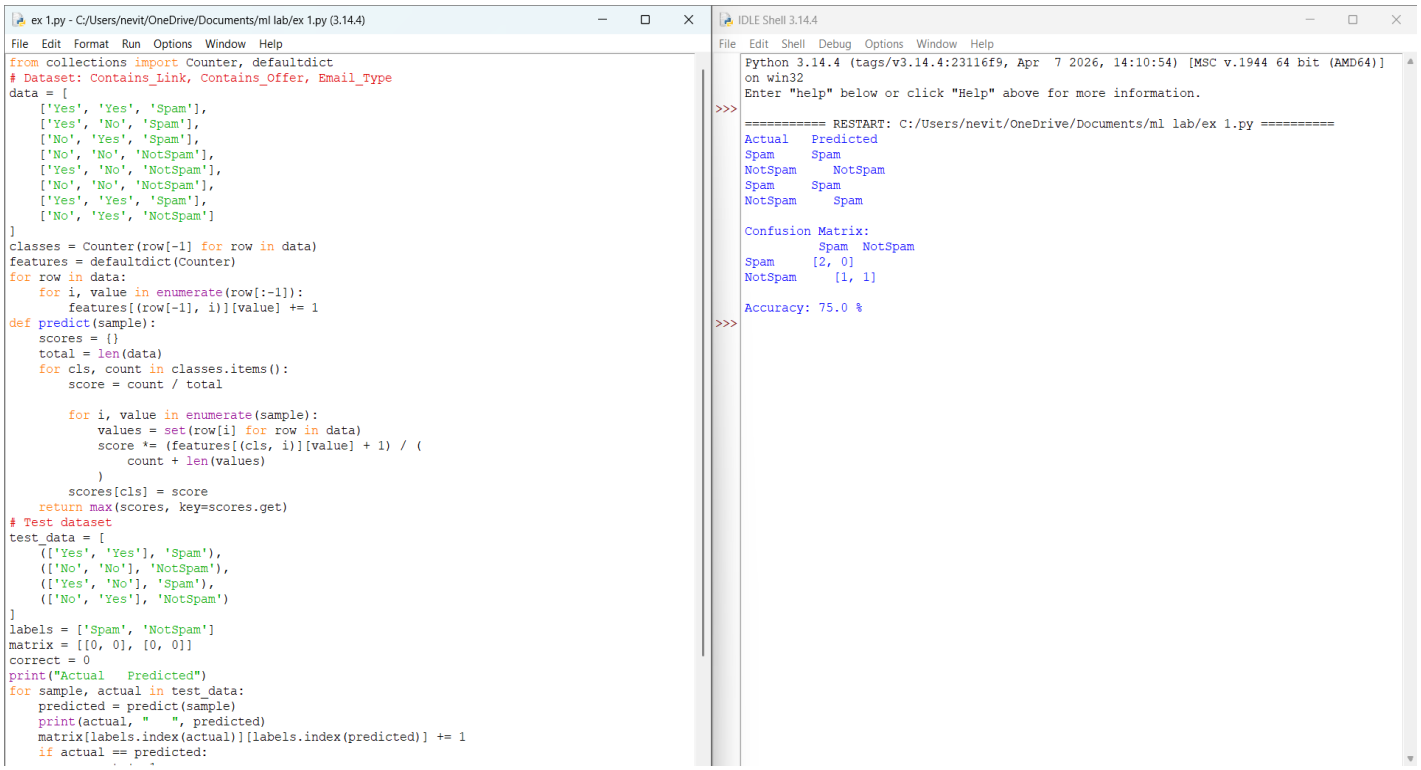
print("Test Sample:", test_sample)
print("K value:", k)
print("Predicted Class:", result)
```

IDLE Shell 3.14.4

File Edit Shell Debug Options Window Help

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py ==========
Test Sample: [165, 60]
K value: 3
Predicted Class: A
>>>
```

6. Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.



```
from collections import Counter, defaultdict
# Dataset: Contains_Link, Contains_Offer, Email_Type
data = [
    ['Yes', 'Yes', 'Spam'],
    ['Yes', 'No', 'Spam'],
    ['No', 'Yes', 'Spam'],
    ['No', 'No', 'NotSpam'],
    ['Yes', 'No', 'NotSpam'],
    ['No', 'No', 'NotSpam'],
    ['Yes', 'Yes', 'Spam'],
    ['No', 'Yes', 'NotSpam']
]
classes = Counter(row[-1] for row in data)
features = defaultdict(Counter)
for row in data:
    for i, value in enumerate(row[:-1]):
        features[(row[-1], i)][value] += 1
def predict(sample):
    scores = {}
    total = len(data)
    for cls, count in classes.items():
        score = count / total

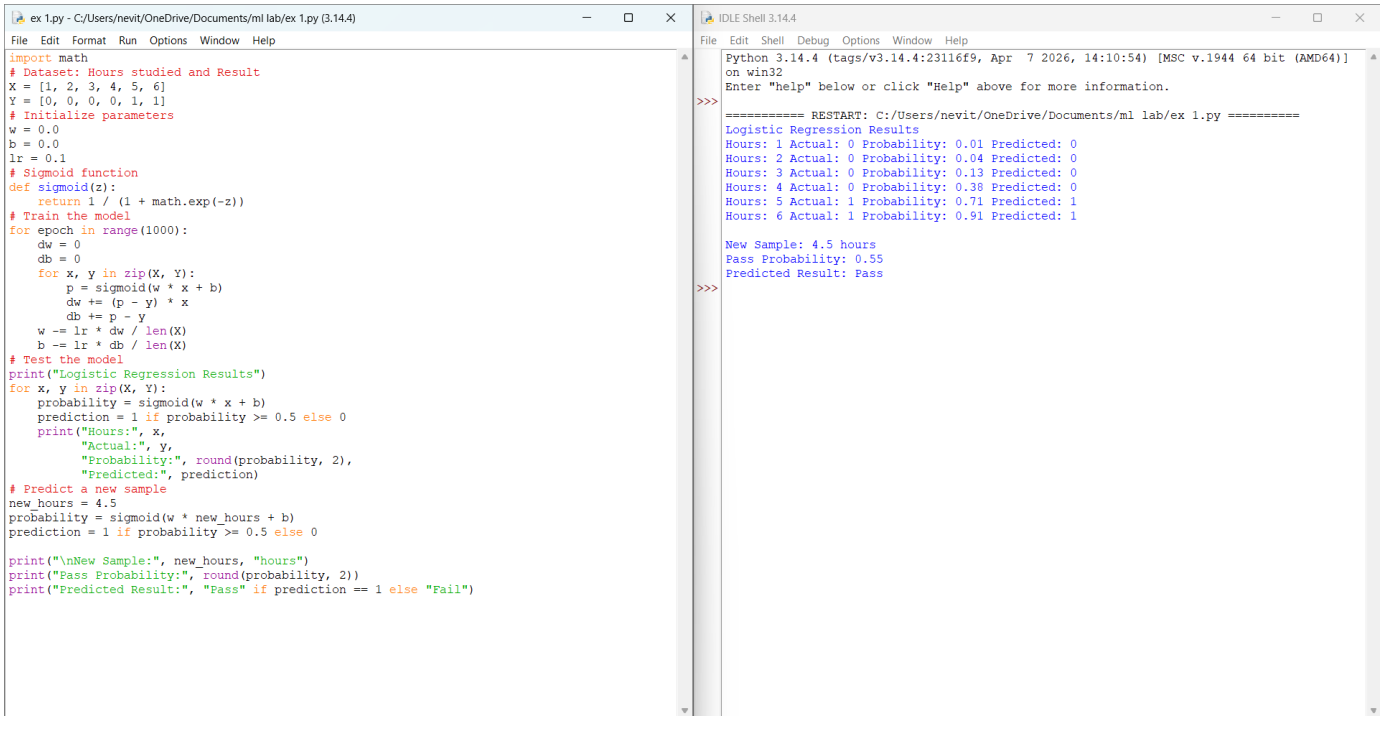
        for i, value in enumerate(sample):
            values = set(row[i] for row in data)
            score *= (features[(cls, i)][value] + 1) / (
                count + len(values)
            )
        scores[cls] = score
    return max(scores, key=scores.get)
# Test dataset
test_data = [
    (['Yes', 'Yes'], 'Spam'),
    (['No', 'No'], 'NotSpam'),
    (['Yes', 'No'], 'Spam'),
    (['No', 'Yes'], 'NotSpam')
]
labels = ['Spam', 'NotSpam']
matrix = [[0, 0], [0, 0]]
correct = 0
print("Actual   Predicted")
for sample, actual in test_data:
    predicted = predict(sample)
    print(actual, "   ", predicted)
    matrix[labels.index(actual)][labels.index(predicted)] += 1
    if actual == predicted:
```

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py ==========
Actual   Predicted
Spam     Spam
NotSpam     NotSpam
Spam     Spam
NotSpam     Spam

Confusion Matrix:
          Spam  NotSpam
Spam     [2, 0]
NotSpam     [1, 1]

Accuracy: 75.0 %
>>>
```

7. Write a program to implement Logistic Regression (LR) algorithm in python



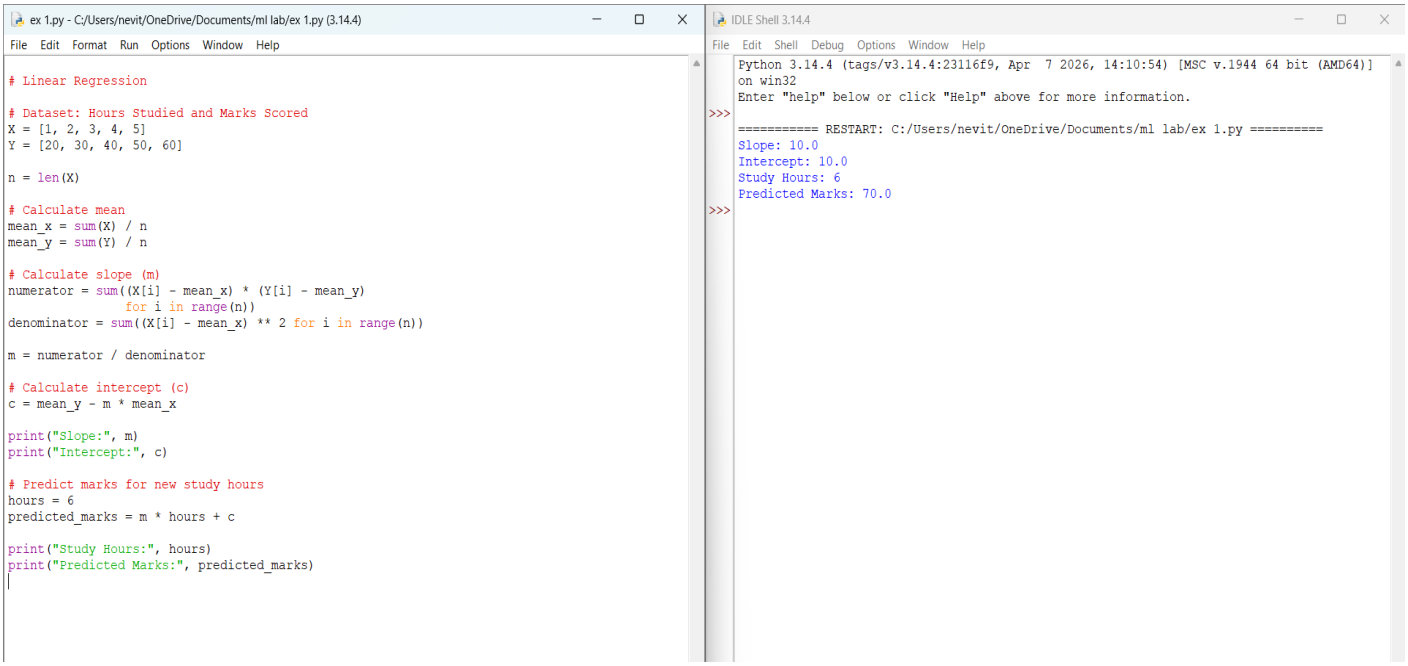
```
import math
# Dataset: Hours studied and Result
X = [1, 2, 3, 4, 5, 6]
Y = [0, 0, 0, 0, 1, 1]
# Initialize parameters
w = 0.0
b = 0.0
lr = 0.1
# Sigmoid function
def sigmoid(z):
    return 1 / (1 + math.exp(-z))
# Train the model
for epoch in range(1000):
    dw = 0
    db = 0
    for x, y in zip(X, Y):
        p = sigmoid(w * x + b)
        dw += (p - y) * x
        db += p - y
    w -= lr * dw / len(X)
    b -= lr * db / len(X)
# Test the model
print("Logistic Regression Results")
for x, y in zip(X, Y):
    probability = sigmoid(w * x + b)
    prediction = 1 if probability >= 0.5 else 0
    print("Hours:", x,
          "Actual:", y,
          "Probability:", round(probability, 2),
          "Predicted:", prediction)
# Predict a new sample
new_hours = 4.5
probability = sigmoid(w * new_hours + b)
prediction = 1 if probability >= 0.5 else 0

print("\nNew Sample:", new_hours, "hours")
print("Pass Probability:", round(probability, 2))
print("Predicted Result:", "Pass" if prediction == 1 else "Fail")
```

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py ==========
Logistic Regression Results
Hours: 1 Actual: 0 Probability: 0.01 Predicted: 0
Hours: 2 Actual: 0 Probability: 0.04 Predicted: 0
Hours: 3 Actual: 0 Probability: 0.13 Predicted: 0
Hours: 4 Actual: 0 Probability: 0.38 Predicted: 0
Hours: 5 Actual: 1 Probability: 0.71 Predicted: 1
Hours: 6 Actual: 1 Probability: 0.91 Predicted: 1

New Sample: 4.5 hours
Pass Probability: 0.55
Predicted Result: Pass
>>>
```

8. Write a program to implement Linear Regression (LR) algorithm in python



ex 1.py - C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py (3.14.4)

File Edit Format Run Options Window Help

```
# Linear Regression

# Dataset: Hours Studied and Marks Scored
X = [1, 2, 3, 4, 5]
Y = [20, 30, 40, 50, 60]

n = len(X)

# Calculate mean
mean_x = sum(X) / n
mean_y = sum(Y) / n

# Calculate slope (m)
numerator = sum((X[i] - mean_x) * (Y[i] - mean_y)
                for i in range(n))
denominator = sum((X[i] - mean_x) ** 2 for i in range(n))

m = numerator / denominator

# Calculate intercept (c)
c = mean_y - m * mean_x

print("Slope:", m)
print("Intercept:", c)

# Predict marks for new study hours
hours = 6
predicted_marks = m * hours + c

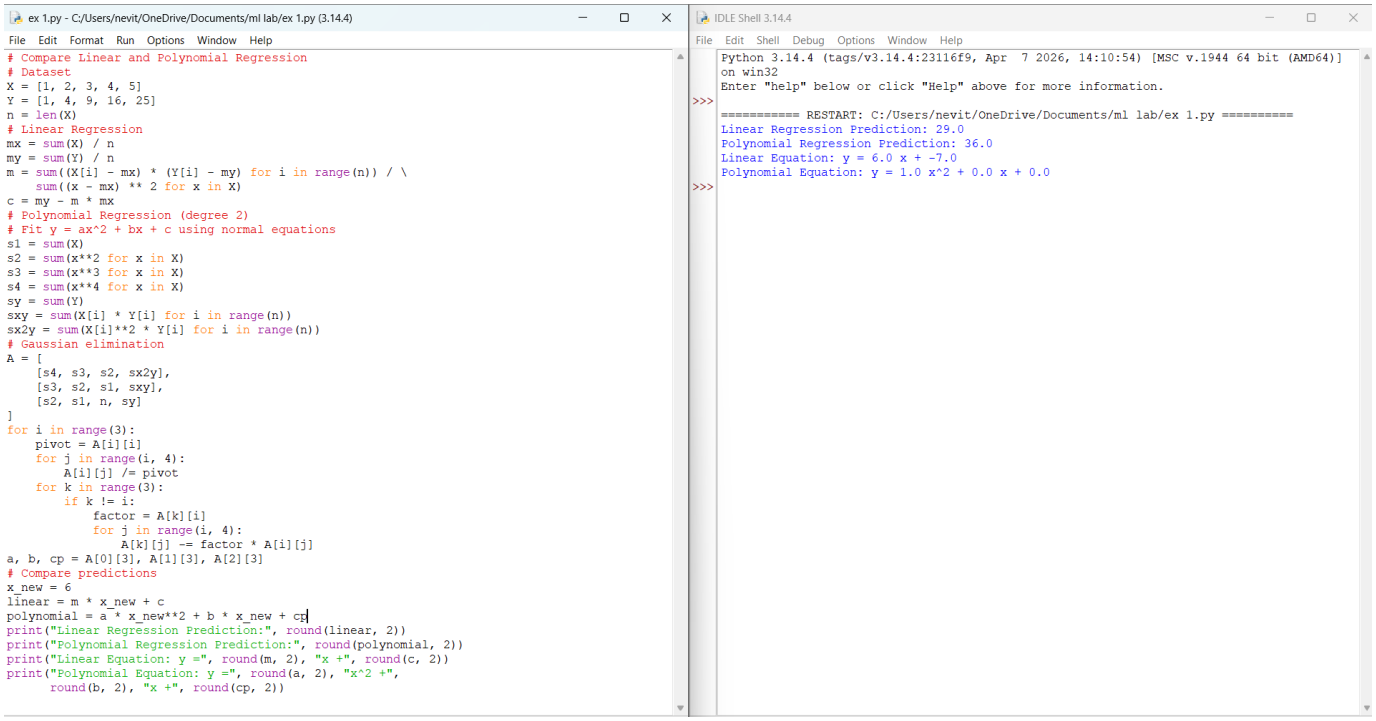
print("Study Hours:", hours)
print("Predicted Marks:", predicted_marks)
```

IDLE Shell 3.14.4

File Edit Shell Debug Options Window Help

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py ==========
Slope: 10.0
Intercept: 10.0
Study Hours: 6
Predicted Marks: 70.0
>>>
```

9.Compare Linear and Polynomial Regression using Python



ex 1.py - C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py (3.14.4)

File Edit Format Run Options Window Help

```
# Compare Linear and Polynomial Regression
# Dataset
X = [1, 2, 3, 4, 5]
Y = [1, 4, 9, 16, 25]
n = len(X)
# Linear Regression
mx = sum(X) / n
my = sum(Y) / n
m = sum((X[i] - mx) * (Y[i] - my) for i in range(n)) / \
    sum((x - mx) ** 2 for x in X)
c = my - m * mx
# Polynomial Regression (degree 2)
# Fit y = ax^2 + bx + c using normal equations
s1 = sum(X)
s2 = sum(x**2 for x in X)
s3 = sum(x**3 for x in X)
s4 = sum(x**4 for x in X)
sy = sum(Y)
sxy = sum(X[i] * Y[i] for i in range(n))
sx2y = sum(X[i]**2 * Y[i] for i in range(n))
# Gaussian elimination
A = [
    [s4, s3, s2, sx2y],
    [s3, s2, s1, sxy],
    [s2, s1, n, sy]
]
for i in range(3):
    pivot = A[i][i]
    for j in range(i, 4):
        A[i][j] /= pivot
    for k in range(3):
        if k != i:
            factor = A[k][i]
            for j in range(i, 4):
                A[k][j] -= factor * A[i][j]
a, b, cp = A[0][3], A[1][3], A[2][3]
# Compare predictions
x_new = 6
linear = m * x_new + c
polynomial = a * x_new**2 + b * x_new + cp
print("Linear Regression Prediction:", round(linear, 2))
print("Polynomial Regression Prediction:", round(polynomial, 2))
print("Linear Equation: y =", round(m, 2), "x +", round(c, 2))
print("Polynomial Equation: y =", round(a, 2), "x^2 +",
      round(b, 2), "x +", round(cp, 2))
```

IDLE Shell 3.14.4

File Edit Shell Debug Options Window Help

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py ==========
Linear Regression Prediction: 29.0
Polynomial Regression Prediction: 36.0
Linear Equation: y = 6.0 x + -7.0
Polynomial Equation: y = 1.0 x^2 + 0.0 x + 0.0
>>>
```

10.Write a Python Program to Implement Expectation & Maximization Algorithm

ex 1.py - C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py (3.14.4)

File Edit Format Run Options Window Help

```
import math
# Dataset
data = [2, 3, 4, 10, 11, 12]
# Initial parameters for two Gaussian clusters
mean1, mean2 = 3.0, 10.0
var1, var2 = 1.0, 1.0
pi1, pi2 = 0.5, 0.5

def gaussian(x, mean, var):
    return (1 / math.sqrt(2 * math.pi * var)) * \
           math.exp(-((x - mean) ** 2) / (2 * var))
# EM algorithm
for iteration in range(10):
    r1, r2 = [], []

    # E-step: Calculate responsibilities
    for x in data:
        p1 = pi1 * gaussian(x, mean1, var1)
        p2 = pi2 * gaussian(x, mean2, var2)
        total = p1 + p2

        r1.append(p1 / total)
        r2.append(p2 / total)

    # M-step: Update parameters
    n1, n2 = sum(r1), sum(r2)

    mean1 = sum(r1[i] * data[i] for i in range(len(data))) / n1
    mean2 = sum(r2[i] * data[i] for i in range(len(data))) / n2

    var1 = sum(r1[i] * (data[i] - mean1) ** 2
               for i in range(len(data))) / n1
    var2 = sum(r2[i] * (data[i] - mean2) ** 2
               for i in range(len(data))) / n2

    pi1 = n1 / len(data)
    pi2 = n2 / len(data)

print("Cluster 1 Mean:", round(mean1, 2))
print("Cluster 2 Mean:", round(mean2, 2))
print("Cluster 1 Variance:", round(var1, 2))
print("Cluster 2 Variance:", round(var2, 2))
print("Cluster 1 Weight:", round(pi1, 2))
print("Cluster 2 Weight:", round(pi2, 2))
```

IDLE Shell 3.14.4

File Edit Shell Debug Options Window Help

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/nevit/OneDrive/Documents/ml lab/ex 1.py ==========
Cluster 1 Mean: 3.0
Cluster 2 Mean: 11.0
Cluster 1 Variance: 0.67
Cluster 2 Variance: 0.67
Cluster 1 Weight: 0.5
Cluster 2 Weight: 0.5
>>>
```